assignment-01\task-2-programming.py

```python
# Task 2 - Programming

# Task 2.1 Unterschied CMU Pronouncing Dictionary versus IPS:
# Bei dem CMU Pronouncing Dictionary http://www.speech.cs.cmu.edu/cgi-bin/cmudict
# unterscheidet sich die Codierung der Phoneme im wesentlichen dadurch von dem
# IPA-System, dass Großbuchstaben des lateinischen Alphabets verwendet werden.
# Bei dem IPA-System wird für jedes Phonem ein eigenes Zeichen genutzt
# (Bsp. ʃ für sch). Das CMU Pronouncing Dictionary ist hauptsächlich für das Englische
# ausgelegt, während das IPA-System international für alle Sprachen genutzt werden
# kann. Das Phonemen-Set im CMU besteht aus 39 Phonemen (des Englischen), aus denen
# die Worte zusammengesetzt werden. Betonungen werden durch die Indizes 0=Ohne Betonung,
# 1=primäre Betonung, 2=sekundäre Betonung abgebildet. Das CMU Pronouncing Dictionary
# nutzt das ARPABET https://en.wikipedia.org/wiki/ARPABET, ein Set phonetischer
# Transkriptions-Codes, die in den 1970er Jahren von der Advanced Research Projects
# Agency (ARPA) entwickelt wurde.
# Bsp: CMU big - B IH1 G, IPA /bɪg/

# Task 2.2 Load pronouncing library
import pronouncing

# Task 2.3 Looking up pronounciation of "Peter put the blue book ..."
sentence = ["Peter", "put", "the", "blue", "book", "beside", "the", "big", "window"]
for word in sentence:
    print(f"{word}: {pronouncing.phones_for_word(word.lower())}") # library needs lower()
print()

# Task 2.4 Write a function that returns True for slant or exact rhymes.
# The Chaos was taken from: https://ncf.idallen.com/english.html
# Erledigt: Write a function are_rhymes(word1, word2) that returns
# True for slant or exact rhymes (e.g., “skate” and “eight”) despite mismatched spellings.
# Prototyp läuft: Extract the nucleus and coda, zeigt True und False für Reimpaare an
# Festlegung "slant", siehe is_slant_rhyme()

def are_rhymes(word1, word2):
    """word1 und word2 sind Reime, wenn sie entweder ein exakter oder ein slant rhyme sind"""
    # vergleicht ob word1 in extrahierter Reimliste von word2 ist
    exact_rhyme = word1 in pronouncing.rhymes(word2)
    slant_rhyme = is_slant_rhyme(word1, word2)
    return exact_rhyme or slant_rhyme

def get_nucleus_and_coda(word: str) -> tuple[str, list[str]]:
    # Beispiel: beside
    # phones: 'B IH0 S AY1 D'
    # rhyming_part: AY1 D
    # nucleus: AY1
    # coda: D
    phones = pronouncing.phones_for_word(word)
    # um später Ergebnis True/False zu verstehen
    print(phones)

    if not phones:
```

```
        return None, None

    # Erste Aussprachevariante verwenden
    rhyming_part = pronouncing.rhyming_part(phones[0]).split()

    if not rhyming_part:
        return None, None

    # Ersten Laut als Nucleus, restliche Laute als Coda
    nucleus = rhyming_part[0]
    coda = rhyming_part[1:]

    return nucleus, coda


def is_slant_rhyme(word1, word2):
    # unsere Regel für slant Reim: gleicher Nucleus und unterschiedliche Coda
    # oder unterschiedlicher Nukleus und gleiche Coda
    nucleus1, coda1 = get_nucleus_and_coda(word1)
    nucleus2, coda2 = get_nucleus_and_coda(word2)

    # Wenn eines der Wörter nicht gefunden wurde
    if nucleus1 is None or nucleus2 is None:
        return False

    return ((nucleus1 == nucleus2 and coda1 != coda2) or
            (nucleus1 != nucleus2 and coda1 == coda2))

word1 = ""                                          # Ablage wort1 initiieren
word2 = ""                                          # Ablage wort2 initiieren
# eigenen lokalen Pfad in input_file eintragen oder hier direkt im NLPROC Ordner
input_file = "assignment-01\\the-chaos.txt"
with open(input_file, "r", encoding="utf-8") as file:
    for line in file:
        if len(line) > 1:                           # Leerzeilen überspringen
            end_word = line.split()[-1]             # letztes Wort holen
            end_word = end_word.strip(".,;:-?!\"'()*")  # entfernt Zeichen links und
rechts
            if word1 == "":                         # letztes Wort Satz 1 ablegen
                word1 = end_word
            elif word2 == "":                       # letztes Wort Satz2 ablegen
                word2 = end_word
                # abgelegtes Wortpaar auf Reim mit Funktion prüfen
                print(word1, word2, are_rhymes(word1.lower(), word2.lower()))
                print()
                word1 = ""                          # Ablage wort1 leeren
                word2 = ""                          # Ablage wort2 leeren

# Bsp.: test für leere Liste bei tortious, Wort gibt es in CMU nicht, daher False
# print(pronouncing.phones_for_word("tortious"))
# print(pronouncing.search(".*tortious.*"))

```